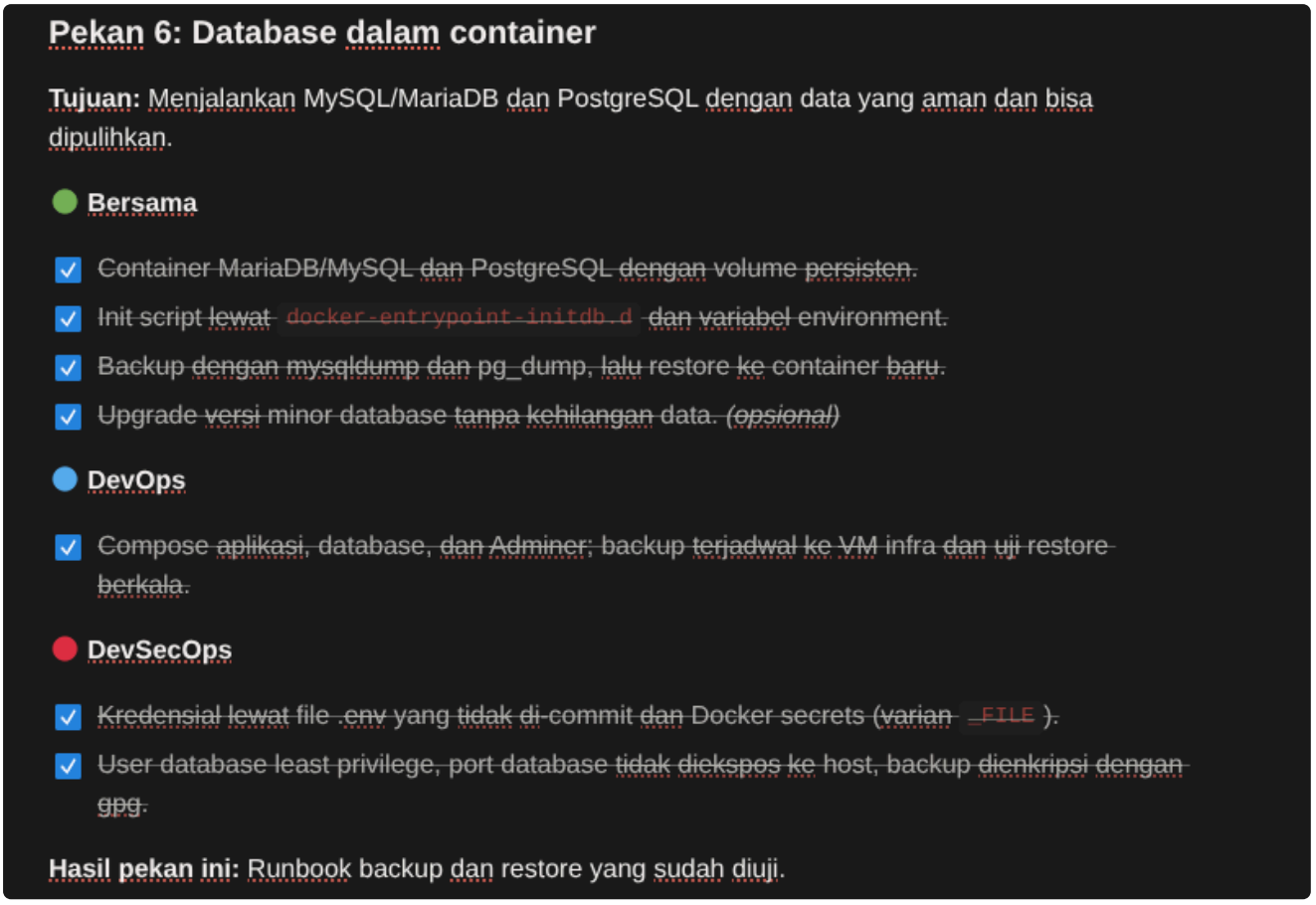
Task 6 - Database Container

Ringkasan Isi

1. Volume Persisten MySQL dan PostgreSQL
2. Membuktikan Volume Tetap Bertahan
3. Init Script
4. Backup dengan mysqldump dan Restore
5. Upgrade Versi Minor Opsional
6. Compose Aplikasi, Database, dan Adminer
7. Backup ke VM Infra
8. Kredensial .env dan Docker Secrets
9. Least Privilege, Port Tidak Diekspose, dan Backup GPG

ini adalah dokumentasi task 6 yang mana task nya seperti ini



Volume Persisten MySQL dan PostgreSQL

dan aku disini ingin memulai langsung dengan membuat container mysql dan postgresql dengan menggunakan volume yang persisten dimana arti persisten sendiri dalam konteks container adalah data yang akan tetap ada atau tidak hilang walau container nya dihapus, direboot atau dibuat ulang,

> **Kenapa harus volume persisten? lalu kenapa harus menggunakan volume yang persisten? ya kalau nginx emang tidak masalah jika kita menggunakan volume persisten karena tugasnya kan basically cuma nerusin file atau suatu benda yang direquest oleh client aja dan kalau database ini spesial karena data yang ada pada volume yang telah ter mount itu setiap hari selalu di write, update, remove, dll dan karena operasi yang dilakukan oleh database ini berbeda, maka disini perlu untuk di mount ke volume tersendiri dengan tujuan menghindari data yang hilang setiap kita mematikan, restart, maupun menghapus dan membuat ulang container nya agar data tetap aman tentram sentosa dan ini hukumnya wajib bagi database untuk dibuatkan volume sendiri.**

MySQL

jadi langsung aja kita menjalankan command untuk bikin yg mysql dulu dengan command

```
docker volume create mysql-data
```

lalu jalankan containernya

```
docker run -d --name db-mysql \
-e MYSQL_ROOT_PASSWORD=rootpass123 \
-e MYSQL_DATABASE=latihan \
-v mysql-data:/var/lib/mysql \
mysql:8
```

PostgreSQL

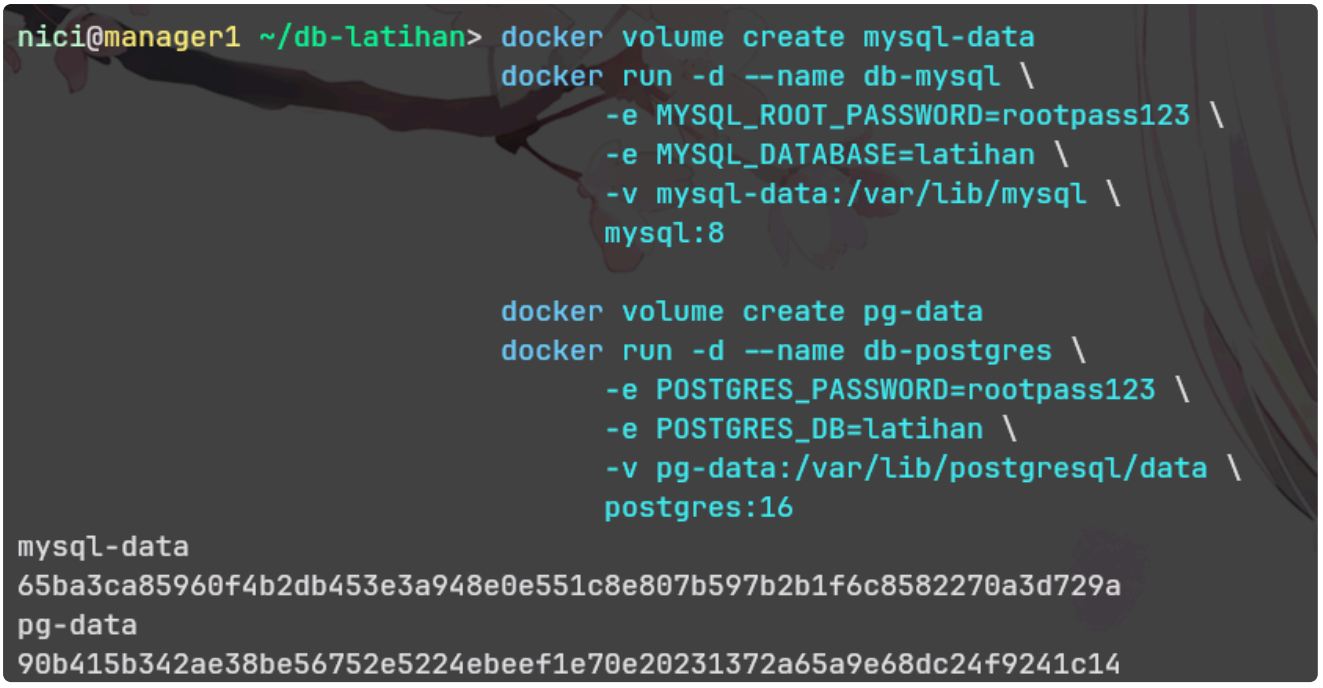
lalu yang postgre kita gunakan command

```
docker volume create pg-data
```

lalu containernya

```
docker run -d --name db-postgres \
-e POSTGRES_PASSWORD=rootpass123 \
-e POSTGRES_DB=latihan \
-v pg-data:/var/lib/postgresql/data \
postgres:16
```

hasilnya kaya gini ya



Membuktikan Volume Tetap Bertahan

terus coba kita akan membuktikan bahwa volume tetap bertahan dengan mengisi data, hapus container, buat ulang, lalu cek apakah datanya masih ada atau tidak dan langsung aja command nya adalah

```
docker exec -it db-mysql mysql -uroot -prootpass123 -e "USE latihan;
CREATE TABLE test (id INT); INSERT INTO test VALUES (1);"

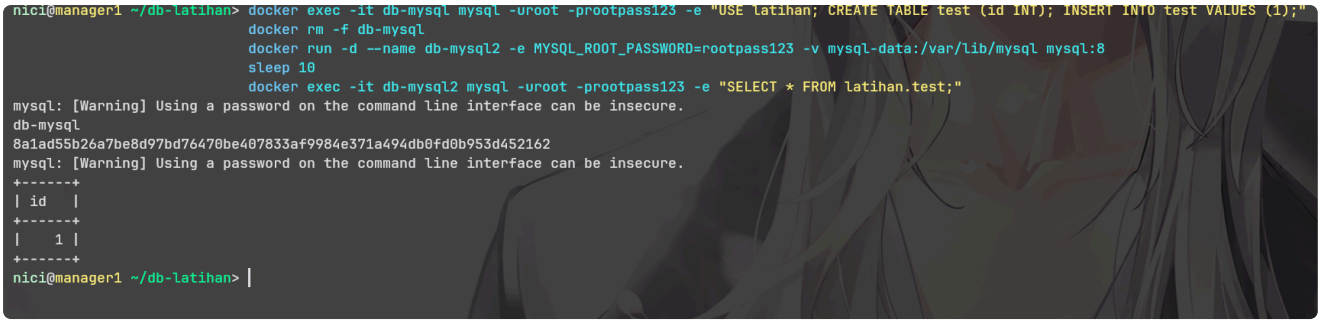
docker rm -f db-mysql

docker run -d --name db-mysql2 -e MYSQL_ROOT_PASSWORD=rootpass123 -v
mysql-data:/var/lib/mysql mysql:8

sleep 10

docker exec -it db-mysql2 mysql -uroot -prootpass123 -e "SELECT * FROM
latihan.test;"
```

hasilnya kaya gini, dan data terbukti masih ada



dan hal yang sama akan terjadi pada postgre juga,

Init Script

next kita akan lanjut ke init script, intinya init script itu mirip dengan init system sejenis systemd yang mana kalau init sejenis systemd kan itu kita menjalankan software2 ke daemon dan kalau init script dalam konteks database ini kita memberi instruksi setelah container dibuat itu database harus ngapain aja, ini sangat membantu untuk proses migrasi karena saat container dibuat, database sudah membuat environment secara otomatis agar database lama langsung bisa menyesuaikan tanpa ada usahan yang ekstra, jadi kira kira gini caranya, pertama kita akan buat direktori khusus lalu nulis script init nya dulu dengan command

```
mkdir -p ~/db-latihan/init
nano ~/db-latihan/init/01-schema.sql
```

lalu isi dengan sintask

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100)
);
INSERT INTO users (name) VALUES ('budi'), ('ani');
```

dan disini kita akan melakukan pembersihan secara menyeluruh pada container beserta volume nya dengan command

```
docker rm -f db-mysql2

docker volume rm mysql-data
```

terus kita juga akan membuat volume baru untuk container yang baru nanti dengan command

```
docker volume create mysql-data
```

terus tinggal jalankan aja command untuk run docker nya

```
docker run -d --name db-mysql \
-e MYSQL_ROOT_PASSWORD=rootpass123 \
-e MYSQL_DATABASE=latihan \
-v mysql-data:/var/lib/mysql \
-v ~/db-latihan/init:/docker-entrypoint-initdb.d \
mysql:8
```

kasi napas dulu dengan command

```
sleep 10
```

terus coba kita exec dan select apakah nama budi dan ani itu ada dengan command

```
docker exec -it db-mysql mysql -uroot -prootpass123 -e "SELECT * FROM
latihan.users;"
```

lalu kira kira hasilnya kaya gini



> ⚠ **Error di Hasil disitu ada error can't connect to database server karena lagi sleep dan disitu aku melakukan exec ulang dan muncul dua data, yaitu budi dan ani**

Backup dengan mysqldump dan Restore

terus kita akan lanjut dengan melakukan backup dengan menggunakan mysql dump ke container baru, disini kita akan ngedump seluruh database ke dalam satu file.sql dan nanti kita akan taruh file hasil dump nya ke container baru dengan command

```
docker exec db-mysql mysqldump -uroot -prootpass123 latihan > ~/db-latihan/backup-mysql.sql
```

lalu kita akan membuat container restore, kita akan namai db-mysql-restore dengan command

```
docker run -d --name db-mysql-restore -e MYSQL_ROOT_PASSWORD=rootpass123 -e MYSQL_DATABASE=latihan mysql:8
```

lalu kita kasih napas dulu dengan command

```
sleep 10
```

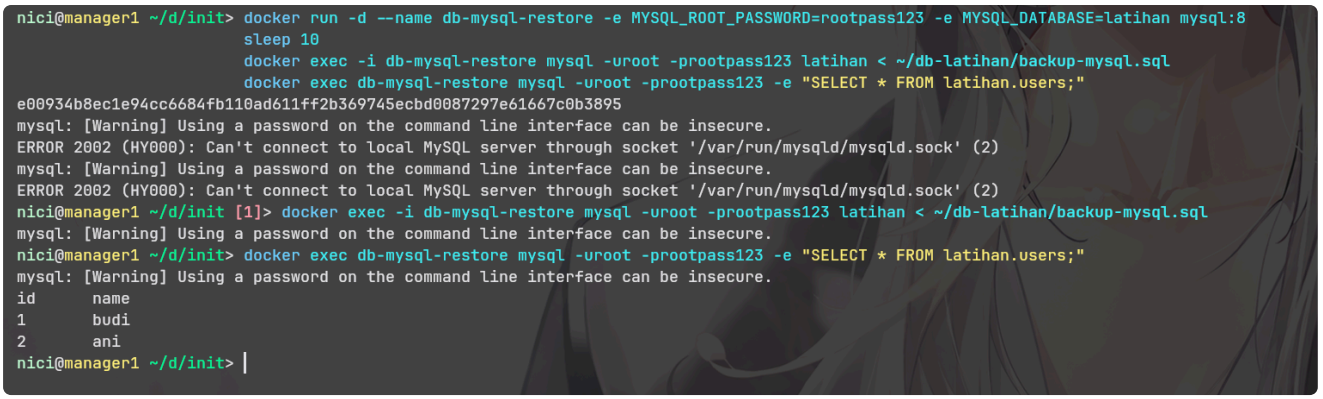
lalu tinggal masukin data hasil backup ke container baru dengan command

```
docker exec -i db-mysql-restore mysql -uroot -prootpass123 latihan < ~/db-latihan/backup-mysql.sql
```

terus tinggal cek lagi dengan command

```
docker exec db-mysql-restore mysql -uroot -prootpass123 -e "SELECT * FROM latihan.users;"
```

harusnya datanya sama dengan container db-mysql dan hasilnya seperti ini



> ⚠ **Error di Hasil seperti biasa, error ada karena sleep tadi jadi kita ketik ulang command nya aja dan ini kebiasaanku untuk membuat command command yang terpisah menjadi satu jadi itu kenapa ada error, kalau dijalankan satu satu gaakan ada error**

Upgrade Versi Minor Opsional

terus next lagi ke upgrade versi minor tanpa kehilangan data, aku skip aja karena opsional tapi kalau mau tau caranya bisa dengan mematikan containernya dulu, remove containernya lalu run ulang dengan menggunakan image yang ada minor update nya, misal kan dengan command

```
docker run -d --name db-mysql \
-e MYSQL_ROOT_PASSWORD=rootpass123 \
-e MYSQL_DATABASE=latihan \
-v mysql-data:/var/lib/mysql \
mysql:8
```

jadi

```
docker run -d --name db-mysql \
-e MYSQL_ROOT_PASSWORD=rootpass123 \
-e MYSQL_DATABASE=latihan \
-v mysql-data:/var/lib/mysql \
mysql:8.xxx(sesuaikan update nya)
```

Compose Aplikasi, Database, dan Adminer

lalu next aja ke ke membuat sebuah compose untuk sebuah aplikasi, database, adminer, dan melakukan backup vm terjadwal yang mana adminer sendiri adalah sebuah webapp ringan untuk kelola database yang kira kira mirip sama phpmyadmin tapi adminer

kompatibel untuk berbagai jenis database sekaligus, langsung aja kita buat direktori dulu dengan command

```
mkdir -p ~/db-compose && cd ~/db-compose
```

lalu langsung aja buat docker compose nya dengan command

```
nano compose.yaml
```

lalu isi dengan

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: rootpass123
      MYSQL_DATABASE: latihan
    volumes:
      - db-data:/var/lib/mysql

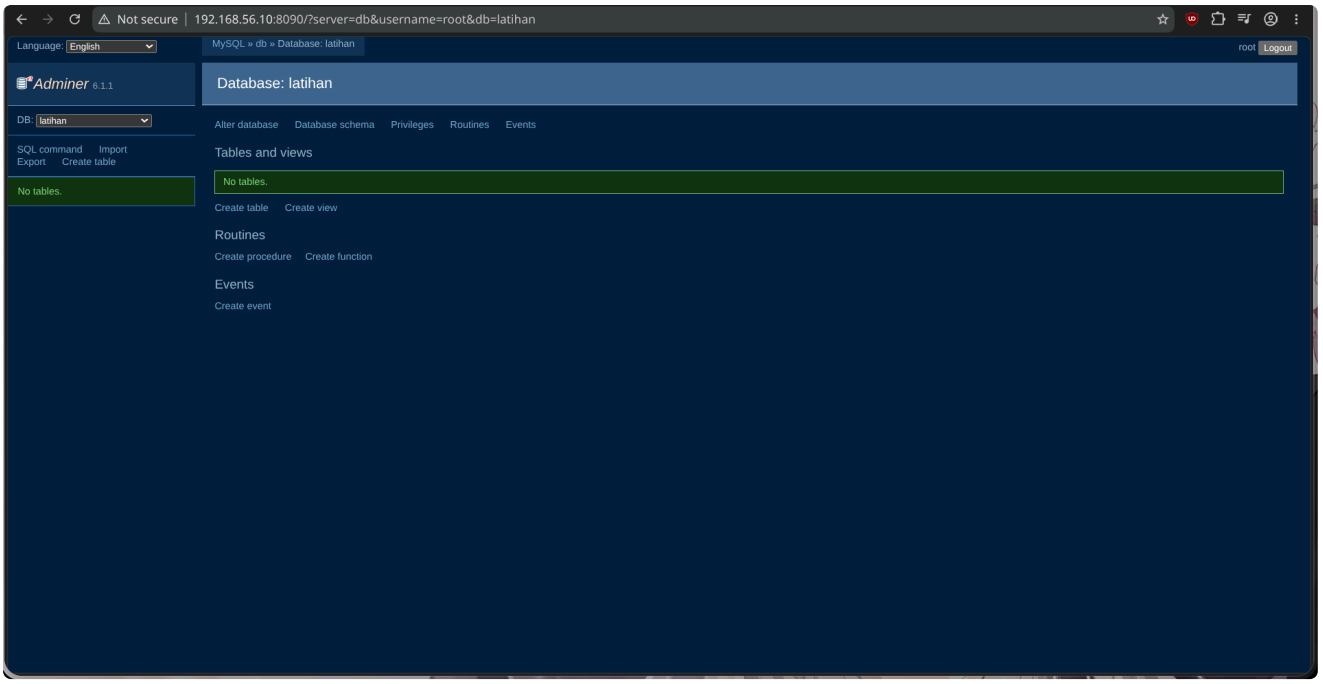
  adminer:
    image: adminer
    ports:
      - "8090:8080"
    depends_on:
      - db

volumes:
  db-data:
```

lalu langsung aja kita jalankan command

```
docker compose up -d
```

setelah itu kita bisa langsung aja login ke adminer pada ip host dari docker nya, yaitu 192.168.56.10 dengan kredensial user adalah root dan password nya rootpass123 dan hasilnya akan seperti ini



Backup ke VM Infra

lalu tinggal kita harus membuat skrip untuk backup ke vm infra dan uji restore berkala dan langsung aja kita buat skripnya dengan comman d

```
nano backup-db.sh
```

lalu isi dengan

```
#!/bin/bash
set -uo pipefail
TIMESTAMP=$(date +%Y%m%d-%H%M%S)
BACKUP_FILE="/tmp/db-backup-${TIMESTAMP}.sql"

docker exec db-mysql mysqldump -uroot -prootpass123 latihan >
"$BACKUP_FILE"
scp "$BACKUP_FILE" nici@infra:~/backups/db/
rm "$BACKUP_FILE"
```

terus jangan lupa kita jalankan command

```
chmod +x backup-db.sh
```

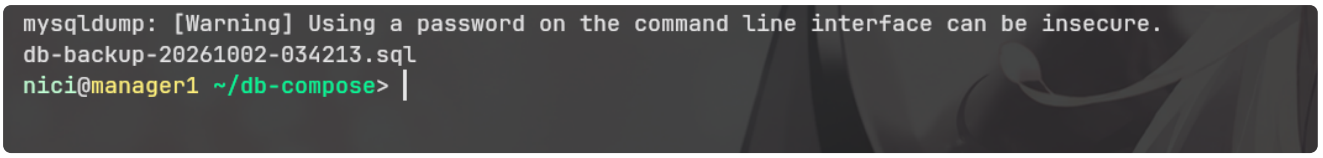
dan sebelum kita menjalankan backupnya, kita harus membuat direktori ~/backups/db/ dulu pada vm infra nya dengan command

```
nano ~/backups/db/
```

lalu kita langsung balik lagi ke vm manager1 dan ke dalam direktori dc-compose dan jalankan

```
./backup-db.sh
```

kira kira hasilnya akan jadi seperti ini



warning nya kita biarkan saja gapapa

Kredensial .env dan Docker Secrets

lalu tinggal masuk ke bagian membuat agar kredensial .env tidak ter commit dan docker secrets menggunakan FILE yang mana langsung ke step by stepnya dengan cara:

masukkan dulu .env nya ke dalam gitignore dengan command

```
echo .env > .gitignore
```

lalu kita akan membuat sebuah kredensial db_password.txt dan isi dengan

```
rootpass123
```

terus setelah itu kita lakukan sedikit modifikasi pada compose.yaml nya dengan mengganti isian compose.yaml dari

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: rootpass123
      MYSQL_DATABASE: latihan
    volumes:
      - db-data:/var/lib/mysql

  adminer:
    image: adminer
    ports:
      - "8090:8080"
    depends_on:
      - db
```

```
volumes:
  db-data:
```

jadi

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_password
      MYSQL_DATABASE: latihan
    volumes:
      - db-data:/var/lib/mysql
    secrets:
      - db_password

  adminer:
    image: adminer
    ports:
      - "8090:8080"
    depends_on:
      - db

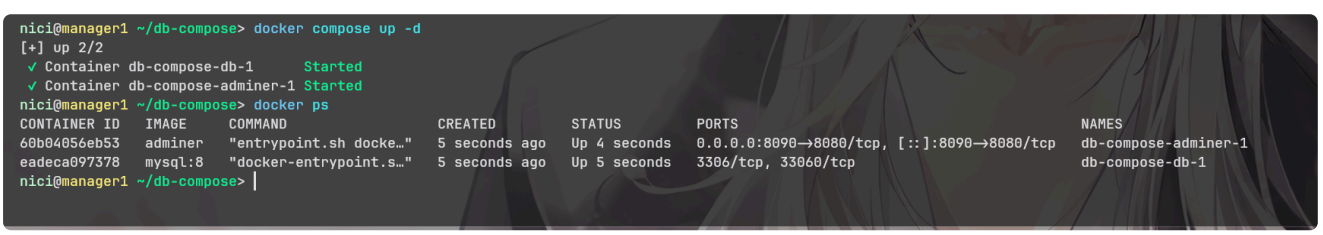
volumes:
  db-data:

secrets:
  db_password:
    file: ./db_password.txt
```

dan langsung aja kita jalankan dengan command

```
docker compose up -d
```

dan ketika dijalankan harunya akan menghasilkan output seperti ini



Least Privilege, Port Tidak Diekspose, dan Backup GPG

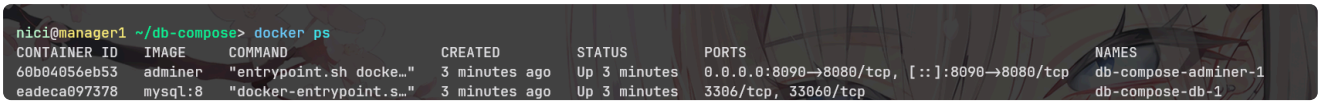
dan yang terakhir kita akan membuat user database dengan least previlegges, port database tidak diekspose ke host dan backup dienkripsi dengan gpg

User dengan Least Privilege

dan kita akan memulai dengan membuat user dengan previlegges yang least, ga setinggi root dengan cara masuk ke mysql dari container nya tapi sebelum itu kita cek dulu container nya dengan command:

```
docker ps
```

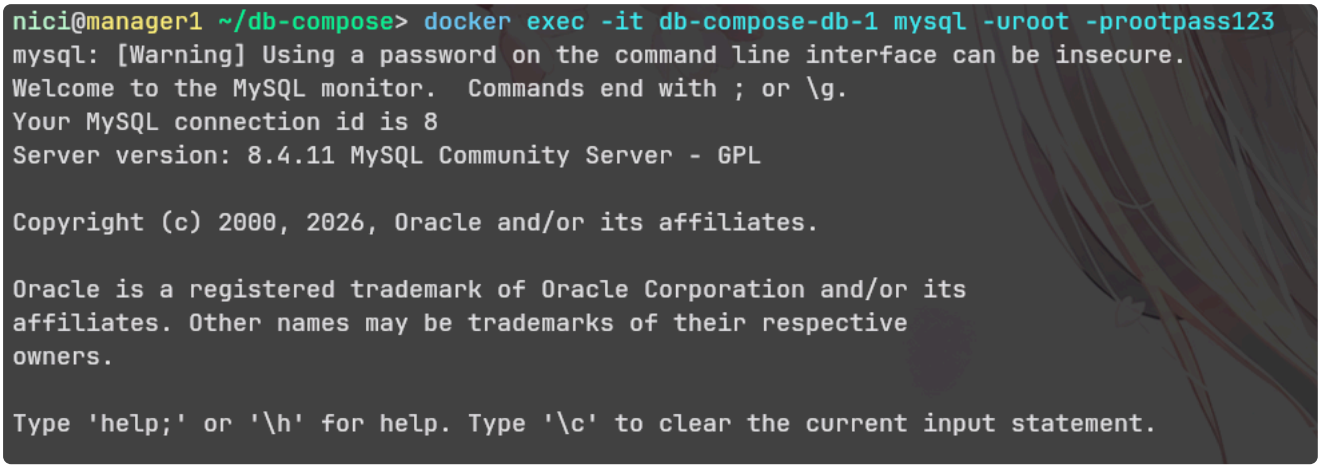
lalu hasilnya seperti ini



terus disitu kita akan login mysql pada container db-compose-db-1

```
docker exec -it db-compose-db-1 mysql -uroot -prootpass123
```

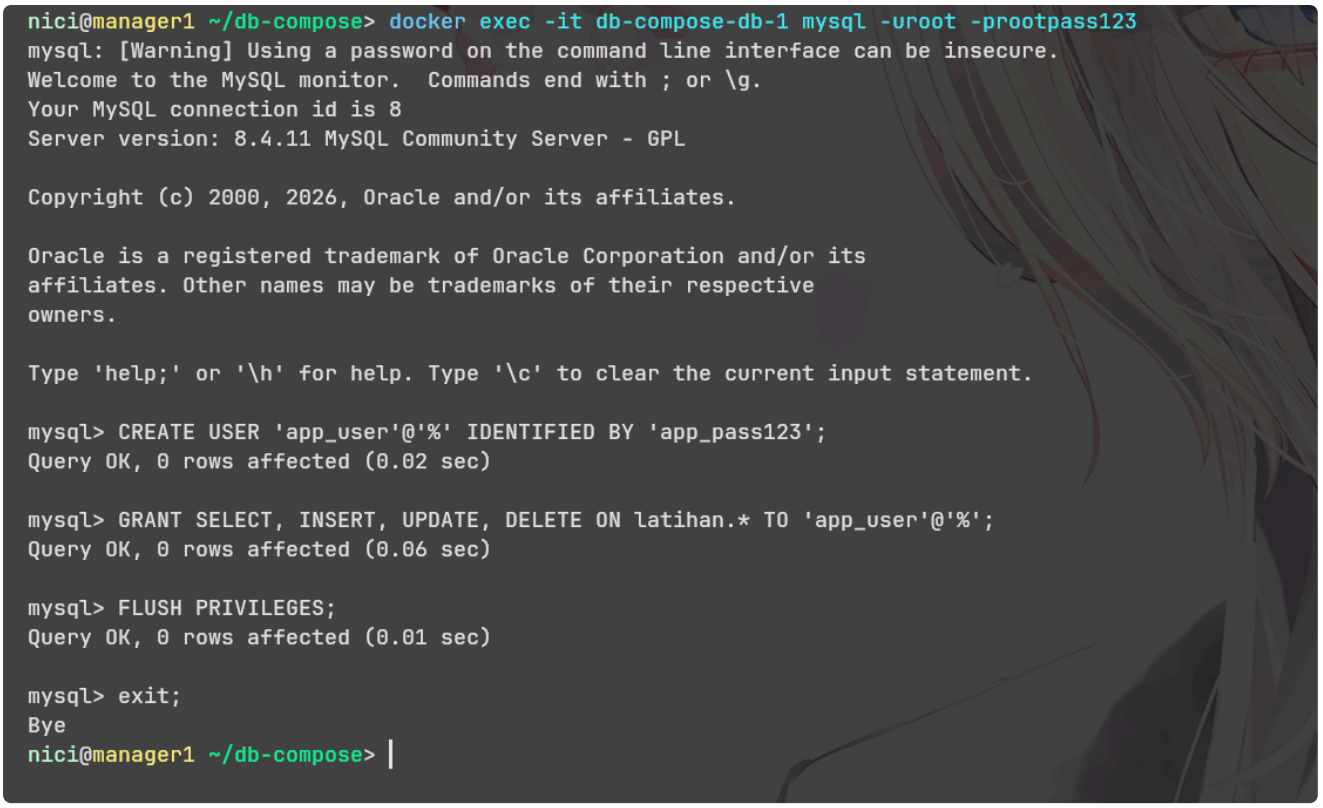
terus kita akan masuk ke shell yang mana kira kira seperti ini



terus kita langsung aja membuat user baru, memberi akses, dan ngeflush dengan command

```
CREATE USER 'app_user'@'%' IDENTIFIED BY 'app_pass123';
GRANT SELECT, INSERT, UPDATE, DELETE ON latihan.* TO 'app_user'@'%';
FLUSH PRIVILEGES;
```

lalu jangan lupa exit; dan setelah itu kira kira hasilnya seperti ini



Port Database Tidak Diekspose

lalu kita juga harus membuat port database tidak terekspose ke host dulu dan untuk melakukan itu kita hasus mematikan container nya dengan command

```
docker compose down
```

lalu kita edit compose.yaml nya dari

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_password
      MYSQL_DATABASE: latihan
    volumes:
      - db-data:/var/lib/mysql
    secrets:
      - db_password

  adminer:
    image: adminer
    ports:
      - "8090:8080"
    depends_on:
      - db

volumes:
```

```
  db-data:

secrets:
  db_password:
    file: ./db_password.txt
```

menjadi

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_password
      MYSQL_DATABASE: latihan
    volumes:
      - db-data:/var/lib/mysql
    expose:
      - "3306"
    secrets:
      - db_password

  adminer:
    image: adminer
    ports:
      - "8090:8080"
    depends_on:
      - db

volumes:
  db-data:

secrets:
  db_password:
    file: ./db_password.txt
```

Perbedaannya lalu apa perbedaannya? kita menambahkan satu konfigurasi expose 3306 untuk memastikan port hanya reachable antar container, tidak ke host

lalu jalankan

```
docker compose up -d
```

dan harusnya hasilnya seperti ini

```
nici@manager1 ~/db-compose> docker compose up -d
[+] up 3/3
 ✓ Network db-compose_default      Created
 ✓ Container db-compose-db-1       Started
 ✓ Container db-compose-adminer-1 Started
nici@manager1 ~/db-compose>
```

Backup Terenkripsi dengan GPG

terus yang terakhir adalah membuat backup yang terenkripsi dengan gpg, langsung aja c

---

Referensi Cepat (tambahan)

Container / Volume	Keterangan
`mysql-data`	Volume persisten MySQL
`pg-data`	Volume persisten PostgreSQL
`db-mysql`	Container MySQL utama
`db-postgres`	Container PostgreSQL
`db-mysql-restore`	Container untuk uji restore dari `mysqldump`
`db-compose-db-1`	Container `db` hasil Compose
`adminer` (port 8090)	Web UI kelola database